Pod Execution Model

How pods ship working software as fast as possible

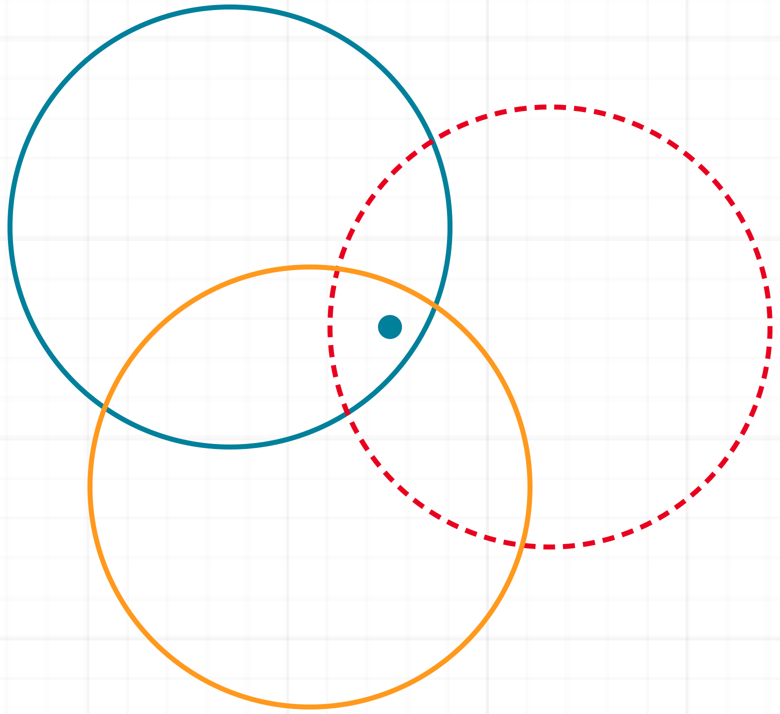
Why Pods Exist

Pods compress and collapse the delivery timeline.

- Small cross-functional strike teams
- Iterate first, not spec first
- Collapse roles, remove translation layers
- AI fills the gaps that used to require more people

Continuous Iteration

Every delivery strengthens the next. The system learns as it runs.



01 Discover

Synthesize stakeholder signals, system context, and competitive inputs to define a differentiated solution direction.

DELIVERABLE: SIGNED-OFF DOCUMENTATION + PROMPT PACKAGE

02 Iterate

Iterate through working Canvas-driven iterations with same-day feedback and pre-code stakeholder alignment. Then build, integrate, and continuously test against acceptance criteria.

DELIVERABLE: VALIDATED ITERATION READY FOR PRODUCTION SCAFFOLDING

03 Launch

Ship to production users, run retrospective, capture learnings, and feed evidence back into the next Discover cycle.

DELIVERABLE: RELEASE + RETRO INTELLIGENCE REPORT

5

Pod Structure

Execution cores, not committees. Two operators drive the pod. Everyone else advises on demand.

CORE OPERATORS

Engineer | Champion

AVAILABLE SUPPORT · ON DEMAND

UX | QA | PM | CS

- Engineer and Champion drive the pod. Everyone else advises.
- AI is the force multiplier. Every person operates at 3-5x output.

Inputs Replace Deliverables

The old model passed documents between roles. The new model uses AI to generate working inputs into a shared iteration.

The Old Way

- ✕ PM writes PRD over days, hands to Design
- ✕ Designer spends weeks in Figma, hands to Eng
- ✕ Engineer builds from spec, hands to QA
- ✕ Each step adds weeks and loses context

The New Way: Roles Collapse

- ✓ Anyone with VS Code can draft, design, or build
- ✓ Champion drafts the problem frame AND helps build a clickable working version
- ✓ Designer generates supporting documentation AND ships clickable UI/visuals.
- ✓ Engineer scaffolds the app while QA agents, kAutomate, and automation generate structured validation coverage
- ✓ No handoffs. Everyone contributes everywhere.

7

Industry Inputs

Feed AI all available context: customer signals, competitive products, internal presentations, articles, and domain expertise. No multi-week spec cycles.

Designer Inputs

UI/UX accelerates refinement after a working iteration, not by defining pixel-perfect specs upfront. Build clickable working versions in a couple hours, not weeks.

9

Engineer Inputs

VS Code + an approved model scaffolds the app. Engineer directs, integrates, and ensures quality.

10

Stakeholder Inputs

QA supports scope definition when needed. Stakeholders remove friction, not approve output.

Artifact Expectations

Every artifact should accelerate testing or iteration. If it doesn't, cut it.

What Pods Produce

- ✓ **Rapid Project Plan** is the live execution artifact pods iterate against
- ✓ Clickable iterations (not static mockups)
- ✓ AI-generated test checklists and edge cases
- ✓ Working demo for stakeholder feedback
- ✓ Short decision notes ("we chose X because Y")
- ✓ Advisors review **Rapid Project Plan** asynchronously when needed

What Pods Don't Produce

- ✕ Multi-page spec documents
- ✕ Polished Figma handoff files
- ✕ Status decks or approval presentations
- ✕ Documentation before the thing works
- ✕ Anything that requires a meeting to review

Rapid Project Plan: First Artifact

The required entry artifact for every pod before work begins.

Problem
What problem are we solving right now?

Primary User
Who runs the workflow?

Definition of Done
Operational · Usable · Demonstrable

Owner
Champion responsible for execution

Scope Boundary
What is explicitly out of scope?

Test Workflow
Exact capture-to-review flow

Success Signal
What proves value quickly?

kahua

12

Rapid Project Plan

This template is actively being modified and standardized. Early example below.

Miami Dade Fast Track

TL;DR

Stabilize File Manager performance for large folder structures by migrating folder permissions to the new domain permission model, then deliver bulk permission management to eliminate folder-by-folder edits.

Success = large projects load without degradation, admins modify permissions across thousands of folders efficiently.

What We're Shipping

- **Backend Architecture:** Migrate to new domain permission model
- **Performance Stabilization:** Eliminate deep folder permission crawl
- **UI Enhancement:** Bulk permission management workflow

Problem Breakdown

Performance

- Deep folder trees degrade performance
- Thousands of folders per project
- API workarounds do not scale

Usability

- Manual edits per folder required
- Miami Dade: ~4000 folders, ~75 groups
- Current UI not viable at enterprise scale

Goal

Ensure File Manager supports enterprise-scale folder structures without performance degradation while enabling efficient bulk permission management.

Scope — Phase 1: Performance

- Replace folder permission logic with new domain model
- Align folder architecture with partition-level permissions
- Validate performance on large data sets

Out of scope: usability enhancements before performance stabilization

Scope — Phase 2: Usability

- Introduce bulk permission editing workflow
- Enable cross-folder updates in a single action
- Ensure solution benefits all customers, not just Miami Dade

Core Technical Decisions

- Adopt onboarding rewrite permission model for folders
- Avoid API-based mass permission mutation
- Performance validation before UI improvements begin

Constraints & Trade-offs

- Euro engineering team owns implementation
- Euro team is resource constrained
- Performance must be solved before usability
- High-visibility customers involved
- No external engineering support for Euro team

Phased Plan

Phase 1: Define migration approach · Assign engineering ownership · Validate performance on large projects
Phase 2: Design bulk permission UX · Implement efficient update workflow · Test across Miami Dade and GSA
Phase 3: Generalize rollout across enterprise · Monitor performance and feedback

Stakeholders

- Engineering Architecture: Colin
- Implementation: Vlad and Euro Team
- API Context: Mike Richie
- Customer Success: Kimberly Day
- Partner Enablement: John Quigley

Pod Execution Vocabulary

Prevents terminology drift across pods.

Pod
A small execution unit built around two core operators and on-demand advisors. Not a traditional team. A focused delivery core designed to move at the speed of decisions.

Advisor
UX, PM, QA, Platform, or any role with relevant expertise. Contributes when the pod needs it. Supports progress rather than gating it. Not standing headcount on any given pod.

Iteration
A working, clickable representation of the workflow. Each one is better than the last. The point is to get something in front of people fast, gather feedback, and improve from there.

Iteration Loop
Discover, Iterate, Validate, Ship, Repeat. The continuous execution cycle pods operate within. The tempo adapts to the problem. Each pass tends to get faster as patterns emerge and become reusable.

Demo
A working demonstration inside the real system, not slides or screenshots. Stakeholder feedback happens here. If it can't be shown working, it likely isn't ready yet.

Champion
The person closest to the problem who drives scope, removes blockers, and makes decisions. Accountable for what ships and whether it solves the right thing. Could be a PM, could be anyone.

Rapid Project Plan
A short execution source of truth. Defines the problem, scope boundary, definition of done, and owner. Intended to be a living document that evolves as the pod learns, not a locked spec.

Inputs
Context gathered from any available source and fed into AI to produce working artifacts. Meeting transcripts, support tickets, competitive screenshots, customer interviews. Replaces sequential deliverables between roles.

Done
Operational and usable. A real user can complete the core workflow. Does not mean polished or fully documented. The goal is shipped capability that can be demonstrated and improved.

Iteration Speed

Continuous operational progress beats delayed polish.

- **Visible progress:** The working iteration advances continuously
- **Stakeholder reaction:** Feedback happens inside the real system, not in review meetings
- **Cross-pod learning:** Successful patterns spread immediately across teams

Definition of Done

Done means operational, testable, usable. Not polished, finalized, documented.

Done Means

- ✓ Operational: works end to end
- ✓ Testable: someone can try it
- ✓ Adoptable: user completes the core flow
- ✓ Demonstrable: showable in a working demo

Done Does Not Mean

- ✕ Pixel perfect
- ✕ Fully documented
- ✕ All edge cases handled

"Teams no longer operate around a PRD.
They operate around an Iteration."

kCapture Pod: Case Study

Floor-plan-anchored inspection capture. Pins hold location history over time. Mobile captures, desktop reviews.

POD COMPOSITION

ProCon Champion | **Colin** Engineer | **Bridgette** UX Advisor

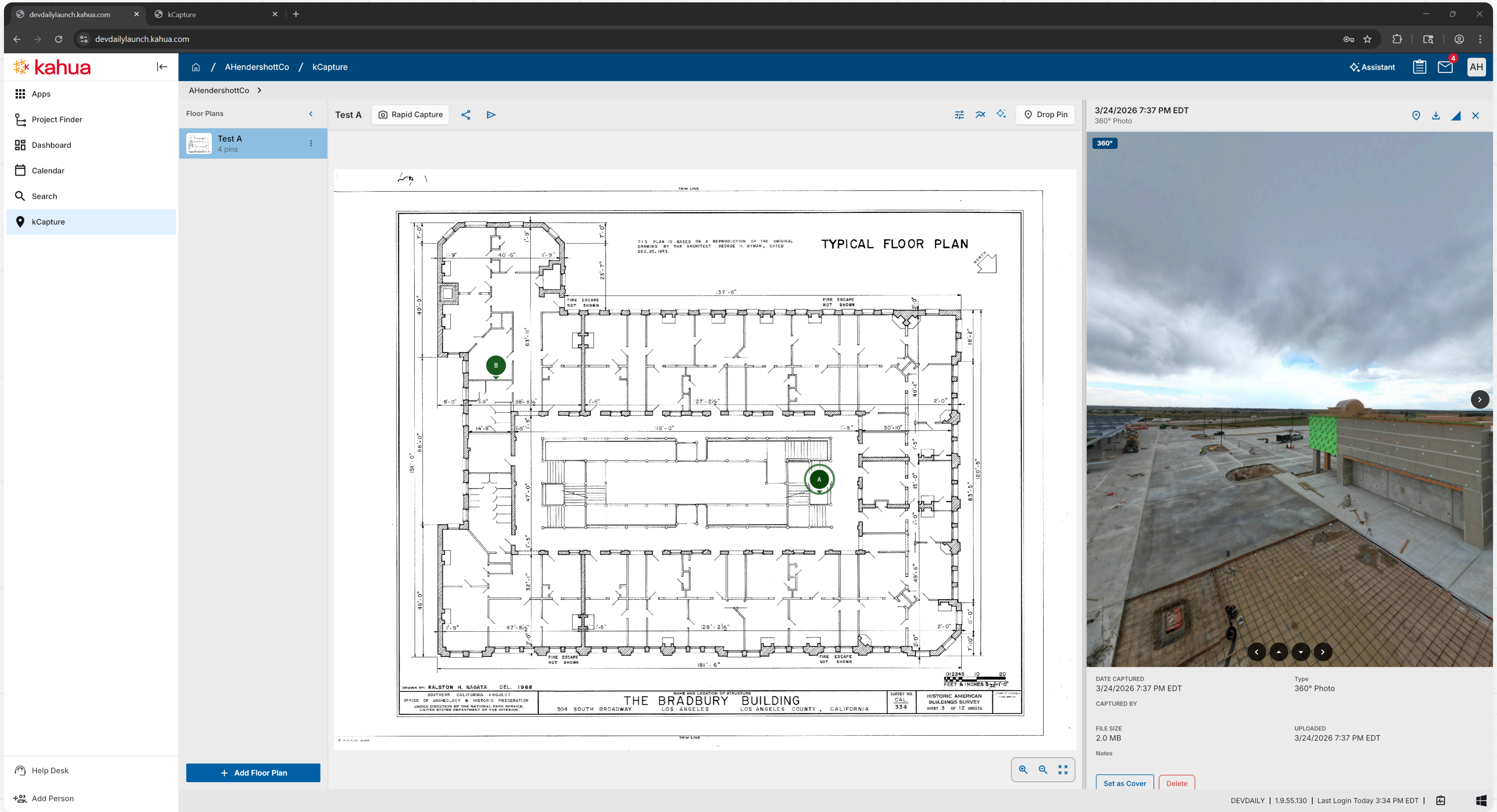
ALREADY SHIPPED

- Floor plans with persistent inspection pins
- Capture timelines per location (photo, video, 360, files)
- Live mobile 360 panorama with guided rotation
- Offline-tolerant upload queue with transfer center
- Side-by-side synchronized comparison across time

WHAT THE POD IS PROVING

- One user can capture, upload, and review without training
- Capture always binds to the correct pin
- Location history is clear on repeat visits
- Sequential pin workflow supports checklist-style inspections

CURRENT DRAFT · MAR 26, 2026 ↓ PDF



Canvas Pod: Case Study

The first pod that proved the model. Canvas shipped as a real product using the execution framework before it was formalized.

POD COMPOSITION

Pritesh Champion | **Colin** Engineer | Sarah UX Advisor

WHAT IT PROVED

- Small team, fast iteration, working product in weeks
- AI-assisted development accelerated every phase
- Stakeholder feedback happened inside the real iteration
- No spec-first pipeline. Problem → iterate → ship

Three Pilot Pods

One execution model. Three proving environments. Each validates a different adoption surface.

22

Timeline

Definition to deployment in rapid cycles. Calendar is the pilot.

- **Now:** Staff the Forward Deployed pod (engineer + designer for Jordan). Confirm all three pod rosters
- **Next:** Education and alignment meeting with everyone in all three pods. Calendar pod kicks off immediately after
- **As Fast As Possible:** Calendar pod ships working product. Model validated. Team can articulate what inputs are actually needed
- **After May Release:** Broader rollout begins pending validation from initial three pods. Calendar pod becomes the template every new team follows

What Changes Now

Concrete behavior shifts starting immediately.

- → No more weeks of polish before learning. Ship unfinished but operational
- → PMs ship a working iteration the same day they write the documentation. Problem frames become working artifacts
- → Designers move into VS Code. Layout ships as working HTML, not Figma files
- → Engineers direct AI, not write every line. Scaffolding, iteration, testing all accelerated
- → QA is everyone's job, embedded from day 1. No downstream handoff
- → Pod execution coordination and artifact tracking owned centrally

APPENDIX

Pod Execution System

The Flow

From problem to product. Four phases. No fixed timeline. Scale to fit.

Discover	Iterate	Build	Launch & Deploy
Understand the problem Stakeholder input, context synthesis, scope definition	**Validate the solution** Clickable iteration, user feedback, stakeholder approval	**Engineer the product** Core flow end-to-end, API integration, continuous testing	**Ship, learn, deploy** Capture learnings, feed next cycle. Regression validation, release deployment, promotion to production *Regression currently partially manual*
OUTPUT: SIGNED-OFF DOCUMENTATION →	OUTPUT: APPROVED ITERATION →	OUTPUT: WORKING BUILD →	OUTPUT: PRODUCTION ●

Who Drives Each Phase

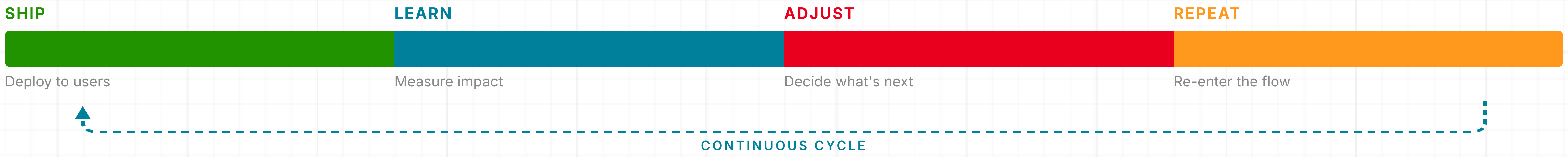
Engineer and Champion are constant. All other roles participate as needed.

	DISCOVER	ITERATE	BUILD	LAUNCH & DEPLOY
Engineer	Tech assessment, integration mapping, env setup	Scaffold real app from iteration patterns	**Core flow end-to-end, API integration, testing**	**Bug fixes, edge cases, regression, deploy**
Champion	**Scope lock, define done, set cadence**	Stakeholder demo, capture feedback, hold scope	Mid-cycle check, kill scope creep, unblock	**Release decision, retrospective, capture lessons**
PM *	**Meetings, synthesize documentation, lock scope**	**Build clickable version, validate with users**	Refine scope, cut non-core, support testing	Soft launch coordination, feedback capture
Designer *	Research, competitor audit, pattern library	**Refine iteration, add all states and flows**	**Polish interactions directly in codebase**	Final interaction polish, launch visuals
QA *	Define test strategy from documentation	Test iteration, shape requirements through bugs	**Continuous testing on real builds, edge cases**	**Regression pass, release sign-off**

** Advisory roles — can be filled by engineer, champion, or on-demand advisor depending on pod composition*

Iteration Mechanisms

The system doesn't end at launch. Every cycle feeds the next.



After every release
Retro within 48 hours. What shipped, what slipped, what we learned.

Continuous feedback
Usage data flows directly into the next cycle. No separate research sprint.

Compounding speed
Patterns become reusable. The pod gets faster. Each cycle builds on the last.

Local Validation Scales to Market Patterns

Clarifies how pods avoid one-off solutions.

- **Validate locally:** Pods validate workflows with real users immediately
- **Aggregate centrally:** Signals aggregate across pods centrally
- **Graduate patterns:** Repeated patterns graduate into platform capabilities
- **Insight through reuse:** Single-customer insight becomes market insight through reuse

PM: Old Way vs New Way

From spec author to decision scientist. The PM's value is judgment, not documentation.

The Old Way

- ✕ Spends weeks writing detailed PRDs before anyone builds
- ✕ Gathers requirements through meetings and documents them in slides
- ✕ Hands finished spec to design, waits for mockups, hands to engineering
- ✕ Success measured by spec completeness, not product learning
- ✕ Manages timelines and Jira tickets, not outcomes

The New Way

- ✓ Defines problem frame inside a live iteration on day one
- ✓ Feeds stakeholder signals, competitive context, and customer data directly into AI to synthesize direction
- ✓ Builds a working clickable version same day using Canvas and approved models
- ✓ Validates assumptions through real user interaction, not spec review
- ✓ Operates as a decision scientist: chooses which problems to solve, not how to document them

Designer: Old Way vs New Way

From pixel pusher to creative director of AI output. Design accelerates iteration, not precedes it.

The Old Way

- ✕ Spends weeks in Figma creating pixel-perfect mockups nobody has tested
- ✕ Delivers static screens with redline annotations for engineering to interpret
- ✕ Design-to-code handoff creates translation loss and rework cycles
- ✕ Feedback happens in comments on static files, disconnected from real behavior
- ✕ Waits for PRD before starting any visual work

The New Way

- ✓ Opens VS Code + an approved model and describes UI in plain language
- ✓ Produces working interactive iterations in hours, testing real states and navigation
- ✓ Directs and curates AI-generated output like a creative director, not a production artist
- ✓ Iterates live alongside engineering in the same codebase, no handoff
- ✓ Refines after a working iteration exists, not before one is built

Engineer: Old Way vs New Way

From line-by-line coder to AI orchestrator. The engineer's value is architecture and judgment, not keystrokes.

The Old Way

- ✕ Waits for spec and mockups before writing a single line of code
- ✕ Builds from scratch, manually writing boilerplate, tests, and API contracts
- ✕ Sprints measured by story points and velocity, not working product
- ✕ Code review is the only quality gate, happens after completion
- ✕ Integration and deployment are separate phases with separate teams

The New Way

- ✓ Takes iteration from PM/Designer and scaffolds production code from patterns on day one
- ✓ Uses AI as a pair partner: generates tests, API contracts, data models, and component variations
- ✓ Primary user flow running end-to-end in the real app within 48 hours
- ✓ Continuous feedback replaces translation cycles. Iteration happens in minutes, not sprints
- ✓ Owns architecture, integration surfaces, and deployment from day one with CI, env config, and feature flags

QA: Old Way vs New Way

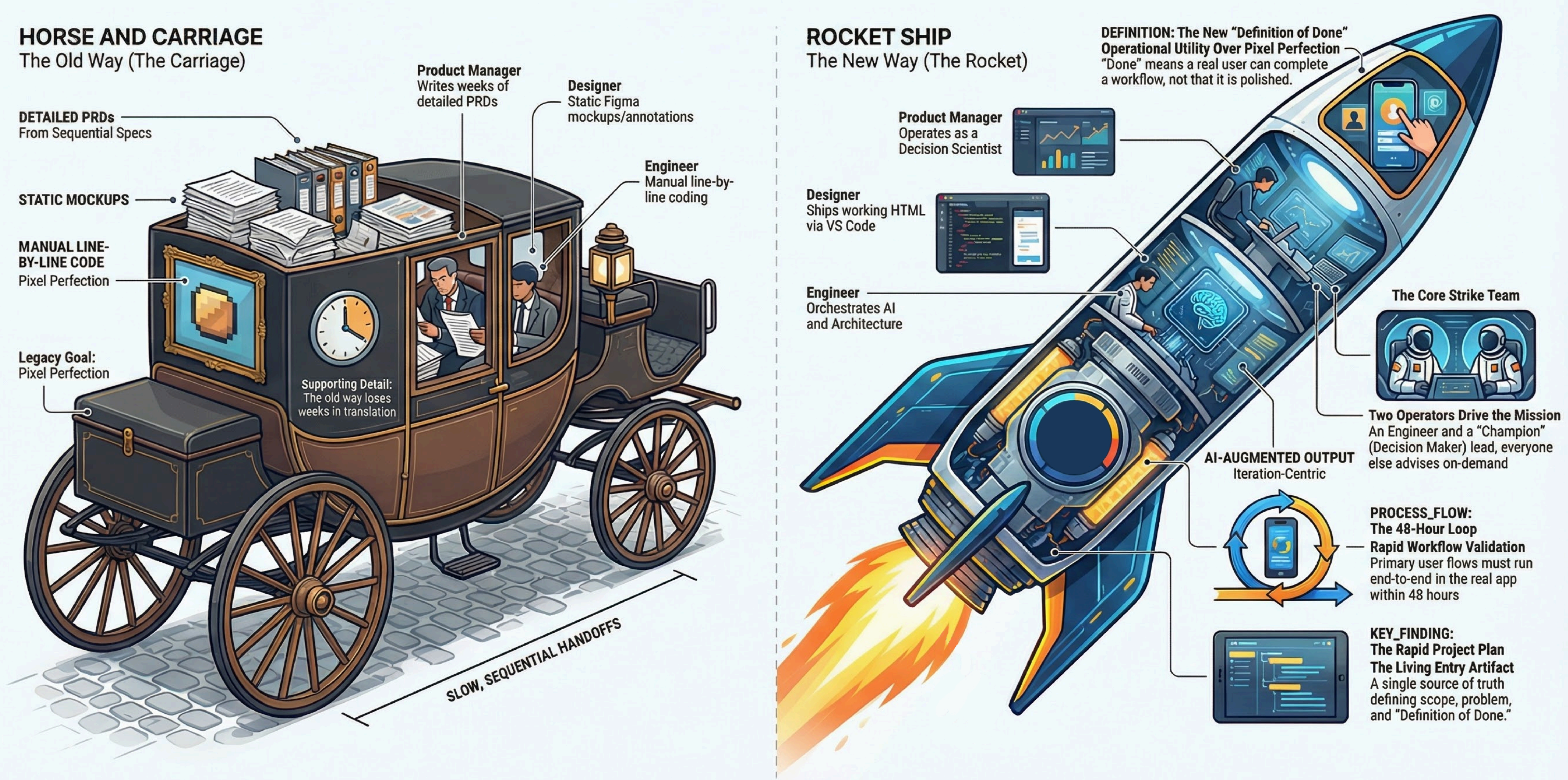
From post-delivery gatekeeper to embedded quality partner. Validation is continuous, not sequential.

The Old Way

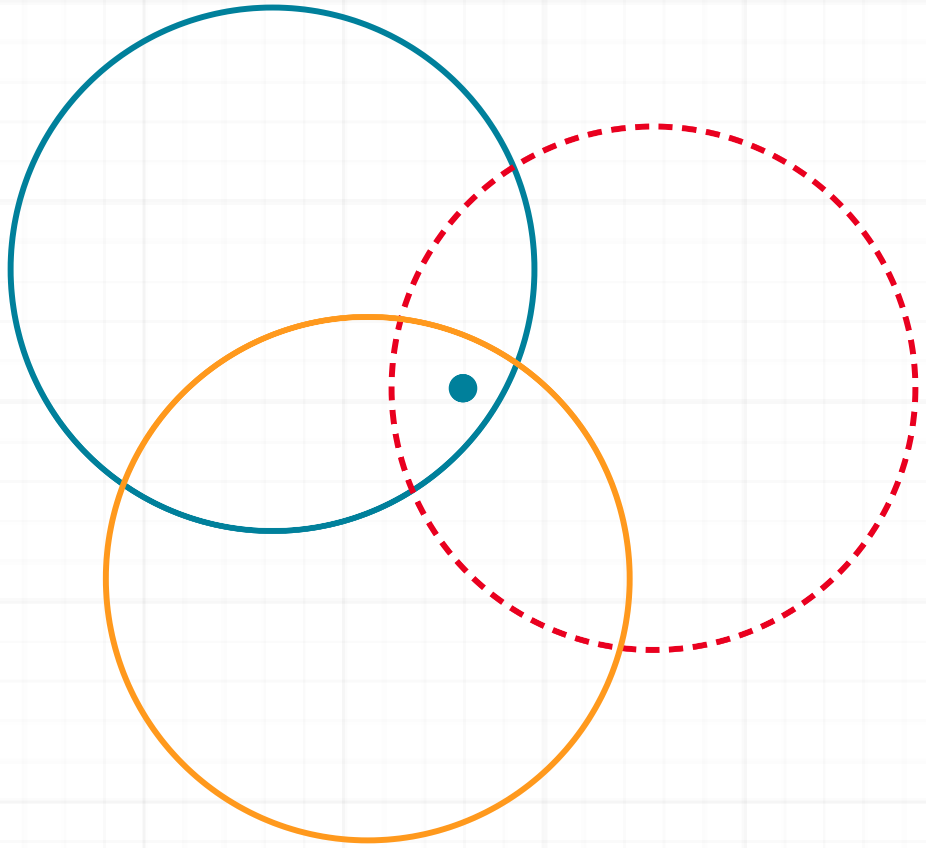
- ✕ Tests after engineering declares "done." Finds bugs after context is lost
- ✕ Writes manual test plans from static specifications
- ✕ QA is a phase: gated, scheduled, and siloed from development
- ✕ Regression testing is manual, slow, and incomplete
- ✕ "Hand off to QA" creates adversarial dynamic and delays release

The New Way

- ✓ Joins at pod kickoff. Tests the first iteration on day one
- ✓ QA agents, kAutomate, and AI generate test scenarios, edge cases, and regression checklists continuously
- ✓ Everyone is QA: PM validates assumptions, Designer tests interactions, Engineer generates automated tests
- ✓ Validates on working iterations, not specifications. Bugs caught in context, not after handoff
- ✓ QA supports pods and accelerates release, never gates it



HORSE AND CARRIAGE
The Old Way (The Carriage)
DETAILED PRDs
From Sequential Specs
STATIC MOCKUPS
MANUAL LINE-BY-LINE CODE
Pixel Perfection
Legacy Goal:
Pixel Perfection
Product Manager
Writes weeks of detailed PRDs
Designer
Static Figma mockups/annotations
Engineer
Manual line-by-line coding
Supporting Detail:
The old way loses weeks in translation
SLOW, SEQUENTIAL HANDOFFS
ROCKET SHIP
The New Way (The Rocket)
DEFINITION: The New "Definition of Done"
Operational Utility Over Pixel Perfection
"Done" means a real user can complete a workflow, not that it is polished.
Product Manager
Operates as a Decision Scientist
Designer
Ships working HTML via VS Code
Engineer
Orchestrates AI and Architecture
The Core Strike Team
Two Operators Drive the Mission
An Engineer and a "Champion" (Decision Maker) lead, everyone else advises on-demand
AI-AUGMENTED OUTPUT
Iteration-Centric
PROCESS_FLOW:
The 48-Hour Loop
Rapid Workflow Validation
Primary user flows must run end-to-end in the real app within 48 hours
KEY_FINDING:
The Rapid Project Plan
The Living Entry Artifact
A single source of truth defining scope, problem, and "Definition of Done."



CURRENT DRAFT

March 26, 2026